



**UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS QUIXADÁ
SISTEMAS DE INFORMAÇÃO**

**PROF. ROBERTO CABRAL
CRIPTOGRAFIA**

**TRABALHO EXTRA
HASH ALVO: 21. 49FC0AA4**

TAILAN DE SOUZA OLIVEIRA

**QUIXADÁ-CE
2025**

SUMÁRIO

1. Introdução e Estrutura do Trabalho.....	3
2. Metodologia de Desenvolvimento e Desempenho.....	4
3. Dificuldades Encontradas.....	5
3.1. Complexidade de Implementação.....	5
3.2. Variabilidade do Tempo e Fator Probabilístico.....	5
4. Resultados Obtidos.....	6
5. Conclusão.....	7
5.1. Análise dos Desafios e Propriedades de Segurança.....	7
5.2. Metodologia de Esforço Computacional.....	7
5.3. Ambiente de Desenvolvimento e Reprodutibilidade.....	7
5.4. Conclusão Final.....	8

1. Introdução e Estrutura do Trabalho

Tudo que for descrito aqui, pode ser melhor observado no github, nesse repositório, lá existe um [readme.md](#) bem detalhado... Além disso, para verificação foi usado o site [CyberChef](#)
<https://github.com/Naliat/TrabalhoExtraCriptografia>

Pensei a princípio, em montar essa arquitetura, para que eu pudesse ao longo do relatório, demonstrar como eu me sai a partir dos testes;

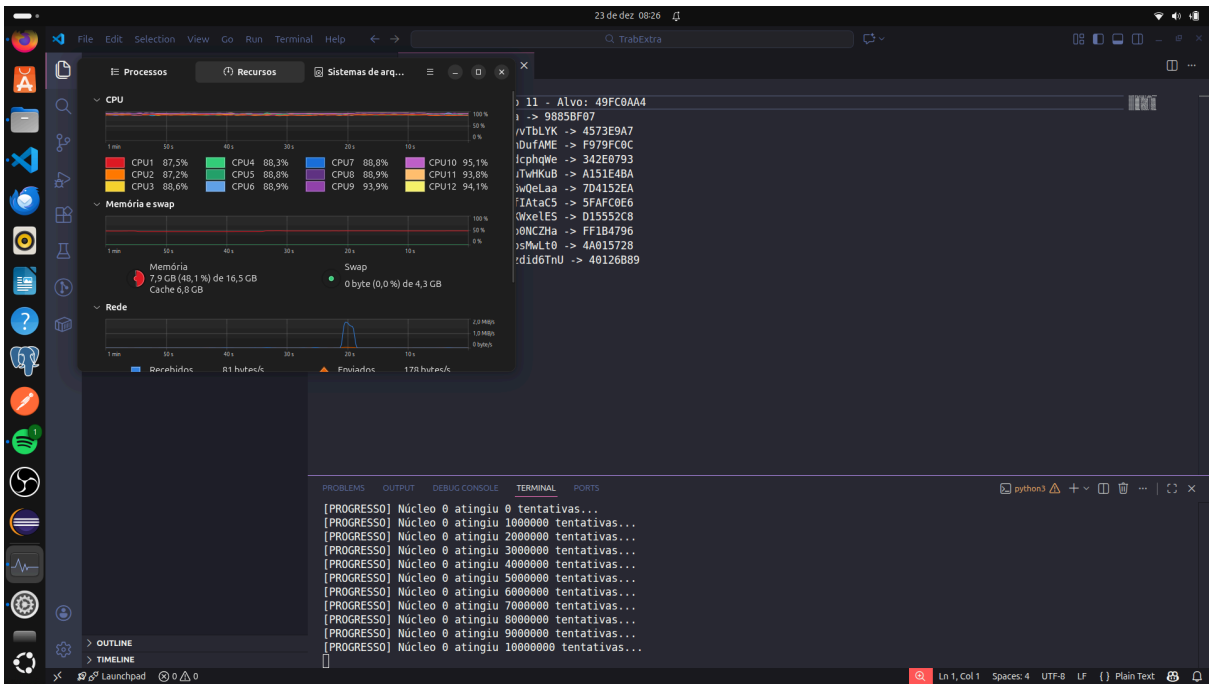
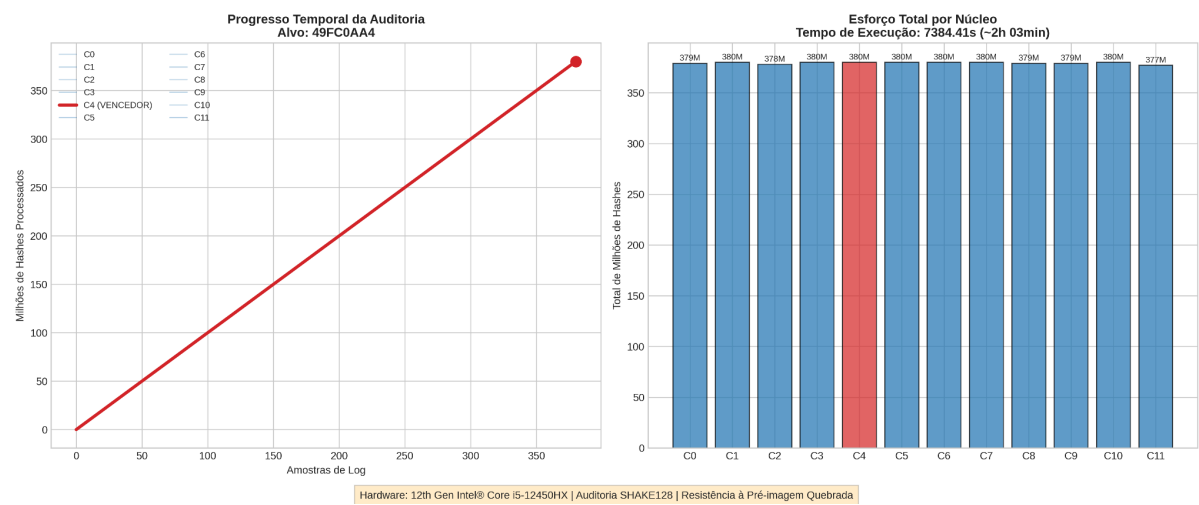
```
|— 🐍 Scripts de Execução (.py)      # Lógica de quebra de hash e geração de gráficos
|— 📁 resultados/                  # Saídas finais, hashes quebradas e o dashboard
|— 📁 testes/                      # Logs brutos de performance por núcleo (0-11)
|— 📁 verificacaoCyberChef/        # Provas de conceito e validações externas
|— 📁 venv/                        # Ambiente virtual Python (isolamento de dependências)
|— 📄 trabalhoExtra.pdf            # Relatório técnico completo
|— 📄 requirements.txt             # Lista de dependências do projeto
```

Este relatório descreve o processo de auditoria das propriedades de segurança da função de hash SHAKE128. A metodologia foi estruturada em três pilares fundamentais:

- Desenvolvimento de Scripts Paralelos: Criação de ferramentas em Python capazes de explorar múltiplos núcleos da CPU para acelerar a busca exaustiva.
- Monitoramento de Desempenho: Implementação de logs e ferramentas de plotagem para visualizar o esforço computacional em tempo real.
- Validação de Resultados: Cruzamento dos dados obtidos com ([CyberChef](#)) para verificar se estava correto.

2. Metodologia de Desenvolvimento e Desempenho

Para medir o desempenho do hardware (i5-12450HX), optei por uma estrutura de visualização baseada em gráficos de linha e barras. Essa escolha permitiu observar como a carga de trabalho foi distribuída entre os 12 threads lógicos e identificar gargalos de processamento (Que foi assim escolhida, a fim de aumentar a rapidez).



3. Dificuldades Encontradas

3.1. Complexidade de Implementação

A maior dificuldade inicial foi compreender a lógica de programação necessária para o multiprocessamento. Foi preciso entender como coordenar diferentes núcleos para que trabalhassem de forma independente, mas sincronizada, garantindo que o programa fosse encerrado imediatamente após o sucesso de um único núcleo através de eventos compartilhados.

3.2. Variabilidade do Tempo e Fator Probabilístico

Uma dificuldade técnica marcante foi a discrepância de tempo observada no Desafio C:

Inconsistência de Testes: Em testes iniciais, o sucesso foi atingido em apenas 8 minutos, enquanto a execução final definitiva levou mais de 2 horas (7384.41s).

4. Resultados Obtidos

Abaixo, os dados finais que comprovam a quebra das resistências:

```
QuebraHash.py desafioA.py resultado_desafio_A.txt X desafioB.py
resultados > resultado_desafio_A.txt
You, 36 minutes ago | 1 author (You)
1  === DESAFIO A: QUEBRA DE RESISTÊNCIA A COLISÃO ===
2  Tentativas: 99419
3  String 1: DGAlk4MrgvS7 | Hash: EC949A40
4  String 2: T3gIp1yaHb0l | Hash: EC949A40
5  
```

```
QuebraHash.py desafioA.py resultado_desafio_B.txt M X desafioB.py
resultados > resultado_desafio_B.txt
You, 2 minutes ago | 1 author (You)
1  === DESAFIO B: SEGUNDA PRÉ-IMAGEM ===
2  x1 (fixo): Aluno: Tailan de Souza Oliveira
3  x2 (encontrado): TI5CfbIHQXQJNnJ
4  Hash Comum: 04AE1ACD
5  Tentativas: 3127297487
6  
```

```
QuebraHash.py desafioA.py resultado.txt X desafioB.py
resultados > resultado.txt
You, 37 minutes ago | 1 author (You)
1  === SUCESSO NA QUEBRA DE PRÉ-IMAGEM === You, 3 hours ago
2  Alvo: 49FC0AA4
3  Candidato encontrado (x): g5wpekdLml1x
4  Hash gerado: 49FC0AA4
5  Núcleo vencedor: 4
6  
```

Download CyberChef

Last build: 5 months ago - Version 10 is here! Read about the new features here

Options About / Support

Operations

Shake

Shake

Parse X.509 certificate

Favourites

Data format

Encryption / Encoding

Public Key

Arithmetic / Logic

Networking

Language

Utils

Date / Time

Extractors

Compression

Hashing

Code tidy

Forensics

Recipe

Shake

Capacity128

Size34

Input

FindReplace

Aluno: Tailan de Souza Oliveira

Output

04ae1acd

Download CyberChef

Last build: 5 months ago - Version 10 is here! Read about the new features here

Options About / Support

Operations

Shake

Shake

Parse X.509 certificate

Favourites

Data format

Encryption / Encoding

Public Key

Arithmetic / Logic

Networking

Language

Utils

Date / Time

Extractors

Compression

Hashing

Code tidy

Forensics

Recipe

Shake

Capacity128

Size34

Input

FindReplace

p8mpOV1WdgnM8q1

Output

04ae1acd

Last build: 5 months ago - version 10 is here! Read about the new features here

Recipe

Shake

Capacity128

Size32

Input

g5wpekdLml1x

Output

49fc0aa4

5. Conclusão

5.1. Análise dos Desafios e Propriedades de Segurança

A auditoria focou na exploração prática das três barreiras de segurança de uma função hash:

- Resistência à Colisão (Desafio A): Através do Paradoxo do Aniversário, comprovou-se que a segurança real de um digest de 32 bits é de apenas 16 bits ($2n/2$), o que permitiu encontrar uma colisão em apenas 1 segundo.
- Segunda Pré-imagem (Desafio B): Diferente da colisão simples, aqui o esforço foi maior (1111 segundos) pois uma das entradas era fixa ("Aluno: Tailan de Souza Oliveira"), impedindo o uso de atalhos estatísticos e exigindo uma busca linear.
- Resistência à Pré-imagem (Desafio C): Representou o cenário de maior dificuldade, onde partindo apenas do hash alvo 49FC0AA4, foi necessário realizar uma busca exaustiva de 34 bits, totalizando mais de 2 horas de processamento intensivo.

5.2. Metodologia de Esforço Computacional

O sucesso da auditoria dependeu diretamente da estratégia de paralelismo. Utilizando 12 threads lógicos do processador i5-12450HX, o trabalho foi distribuído para garantir que bilhões de hashes fossem processados por segundo.

- Monitoramento: O desempenho foi registrado em arquivos de log e visualizado através de dashboards gerados com matplotlib e pandas, permitindo identificar o momento exato em que o núcleo vencedor encontrou o candidato.

5.3. Ambiente de Desenvolvimento e Reprodutibilidade

Para garantir que os resultados possam ser auditados por terceiros, o ambiente foi isolado utilizando as versões específicas das bibliotecas listadas no README.md.

- A biblioteca pycryptodome foi a base para a implementação do SHAKE128, enquanto o numpy auxiliou na gestão dos dados de performance.
- A validação externa foi realizada via CyberChef, confirmando que os candidatos encontrados (pBmpOviWdgnM8q1 e g5wpekdlml1x) geram os hashes alvos conforme as especificações do NIST para a família SHA-3

5.4. Conclusão Final

A auditoria concluiu que digests de 32 e 34 bits são insuficientes para garantir a segurança em sistemas modernos. Ficou evidente que:

1. Hardware comum é capaz de quebrar essas proteções em tempos curtos (minutos a poucas horas).
2. O fator probabilístico influencia o tempo de descoberta, variando conforme a posição do candidato no espaço de busca.
3. A escala de bits é crucial: o acréscimo de apenas 2 bits no Desafio C aumentou o esforço computacional em ordens de grandeza em relação ao Desafio B.